CS200 Introduction to Programming

Instructors (Section1): Shafay Shamail, Saqib Ilyas, Zartash Uzmi

Lab# 06: Object and classes

Lab Guidelines

1. You are allowed to perform/submit the lab only during the lab timings.
2. Make sure you do not leave the lab without submitting your work on LMS.
3. Put all .c files into a folder **YourRollNo_Lab6_TAname** and submit it on LMS (Assignments > Lab02).
4. Note that the submission tab closes at the same time when the lab ends.
5. Please make sure that you have gone through the Lab Manual before starting the lab.
6. Talking to each other is NOT permitted. If you have a question, ask the lab assistants.
7. The objective is not simply to get the job done, but to get it done in the way that is asked for in the lab.
8. Copying/sharing code is strictly prohibited. Using any unfair means will lead to immediate disqualification.
9. Any cheating case will be reported to Disciplinary Committee without any delay.

Programming Conventions

1. Use tabs instead of spaces.
2. Variable names are lowercase.
3. Constant names are uppercase.
4. Function names are first word small, all other words camel case. For example, thisIsAFunction().
5. There are spaces after reserved words and between binary operators.
6. Braces must be vertically aligned.
7. No magic numbers may be used.
8. Every function must have a comment.
9. At most 30 lines of code may be used per function.
10. Global variables are not allowed unless specifically instructed.
11. Use of *goto* statement is not allowed.

Lab2 Marks Distribution

Total marks
100

Move to the next page to view the lab tasks.

Let's Begin …

Note:

- **Plan Your Design:**
  Sketch out class diagrams or flowcharts if necessary. A clear design will help you implement functions that interact smoothly.

- **Problem Solving:**
  While the manual provides detailed requirements, it does not prescribe every implementation detail. You are expected to make reasonable design choices and justify them in your documentation if needed.

Implementation Guideline

- **Input Validation:**
  Your functions must check for the following invalid inputs:

  - **Ensure price of any menuItem cannot be below (Rs) 100. Any price below this value would be initialized to the base cost (Rs. 100)**
  - **Customer's contact number must be strictly a numerical value (For e.g. values like "xyz3214425678" will not be saved. The system would rather initialize it to "N/A"**
  - Ensure discount is a valid percentage between 0.0 and 60.0 (maximum allowed discount percentage). An invalid percentage outside this range will lead to no discount being applied.
  - Ensure orderServeTime() correctly formats and handles cases where the estimated time goes beyond midnight since Zaan runs after midnight too on popular student demand :) **(e.g. currentTime 23:45 and 45 mins total order prep time will give "00:30")**

- **Encapsulation and Modularity:**
  Ensure that class attributes that are private are manipulated only through public member functions. Use constructors to enforce proper initialization of your objects.

- **Destructors:**

Ensure each destructor prints a custom message.

Testing and Expected Behavior

You have to implement all the functions given in the solution.cpp and Here you are provided with the Test.obj file instead of Test.cpp. So to run your test cases you have to execute the following commands:

```
g++ solution.cpp test.obj -o solution.exe
```

then

```
.\solution.exe
```

The provided *Test.obj* file runs a series of tests that simulate:

- Basic MenuItem operations (intialization, constructors, destructor)
- Basic Customer operations
- Basic Order operations (adding and removing items, checking if order full)
- Error handling measures
- Order splitting
- Applying discounts on total order price
- Removing duplicates from orders
- Manual time computation to calculate correct orderserveTime
- Enforcement of menu order limits
- Generation of detailed order receipts

Before submission, ensure your implementation passes all tests —the focus is on your ability to apply object-oriented design principles and handle edge cases.

Task 1: Zaan Menu Ordering System **100 marks**

System Overview

The application is divided into three main components:

1. **MenuItem** – Represents an item available on the restaurant menu.
2. **Customer** – Represents a customer placing an order.
3. **Order** – Manages details of an order including items, price, and estimated preparation time.

Each component is defined in its own header file (e.g., *MenuItem.hpp*, *Customer.hpp*, *Order.hpp*), and your implementations must satisfy the requirements set by the test cases.

Detailed Requirements

1. MenuItem class (MenuItem.hpp) (10 Marks)

The `MenuItem` class is a lightweight object used to record the details of a single Menu Item.

Given Attributes

- **itemName** (string; name of the menuItem e.g. “Burger”)
- **prepTime** (int; estimated preparation time in minutes)
- **price** (double; price of the menuItem)

Required Member Functions

- **MenuItem()**
  Purpose: Default constructor that initializes **itemName** to “N/A” , **price** to 0.0 and **prepTime** to 0.

- **MenuItem(string name, double price, int time)**
  Purpose: Parameterized constructor.
- **MenuItem(const MenuItem &other)**
  Purpose: Copy constructor.
- **void setItemName(string name)**
  Purpose: Sets the menu item name.
- **void setPrice (double price)**
  Purpose: Sets the item price.
- **Void setPrepTime(int time)**
  Purpose: sets the preparation time.
- **Getter functions also**
- **Destructor: Prints a message when MenuItem object is deleted**

2. Customer Class (Customer.hpp) (15 Marks)

The `Customer` class encapsulates the details of an individual bank account.

Attributes (all attributes are private)

- **customerName** (string; name of the Customer)
- **contactNumber** (string; Contact number of the Customer)

Required Member Functions

- **Customer()**
  Purpose: Default constructor that initializes **customerName** to "N/A" and **contactNumber** to "N/A".
- **Customer(string name, string number)**
  Purpose: Parameterized constructor.
- **Customer(const Customer &other)**
  Purpose: Copy constructor.
- **void setCustomerName(string name)**
  Purpose: Sets the customer's name.
- **void setContactNumber (string number)**
  Purpose: Sets the customer's Contact Number. (It should only contain digits. Handle all alphanumeric edge cases)
- **string getCustomerName()**
  - **Purpose:** Returns the customer's name.
  - **Return:** The customerName as a `string`.
- **string getContactNumber()**
  - **Purpose:** Returns the customer's contact number.
  - **Return:** The contactNumber as a `string`.
- **Destructor:**
  - **Prints a message when Customer object is deleted**.

3. Order Class (Order.hpp) (75 Marks)

The `Order` class is responsible for maintaining a collection of Menu Items and coordinating operations for a single order.

Attributes (all attributes are private)

- **orderTable** (int; unique order table number)
- **customer** (Customer; customer placing the order)
- **items** (Array of **menuItem** objects)
- **itemCount** (int; tracks the number of items in the order)
- **static const int maxItems = 10** (maximum number of items per order)

Required Functions (Constructors + Destructors = 10 marks)

- **Constructor: Order()**
  - **Purpose:** Default constructor. Initialize the Order.
  - **Notes:** Ensure that all internal data members apart from the one already initialized, are properly initialized.
- **Order(int n, Customer c)**
  - **Parameters:**
    - `n`: A unique identifier for Order Table number.
    - `c`: Customer placing the order.
  - **Purpose:** Parameterized constructor to initialize order table number and customer. (Remember to initialize `itemCount` as well!)
- **Order(const Order &other)**
  - Copy constructor.
- **bool addItem(MenuItem item) (5 marks)**
  - **Parameters:**
    - `item`: The name of the menu item.
  - **Purpose:** Adds a menu item to the order.
  - **Return:** `true` if successful; `false` if order has reached its capacity.
- **bool removeItem(string itemName) (5 marks)**
  - **Parameters:**
    - `itemName`: The name of the item to be removed from the order.
  - **Purpose:**
    - Removes the item with the given itemName from the order.

        - Updates `itemCount` accordingly.
    - **Return:** `true` if item is removed successfully; `false` otherwise.
- **void removeDuplicates() (10 marks)**
    - **Purpose:** Facilitate the order log by removing duplicate order items. The function must:
        - The first occurrence of each item should be retained.
        - Subsequent occurrences of duplicate menu items should be deleted.
        - After removing duplicates, `itemCount` should be updated accordingly.
- **bool isOrderFull() (5 marks)**
    - **Purpose:** Checks if an order has reached its item limit.
    - **Return:** `true` if limit has reached; `false` otherwise.
- **double calculateTotalPrice(double discount) (5 marks)**
    - **Parameters:**
        - `discount:` the discount (in percentage) applicable (valid values are from 0 till 60.0 only; a discount value greater than 60.0 is disregarded and no discount is applied)
    - **Purpose:** Computes the total price of all the items in the order and applies the discount (if non-zero) on the price.
    - **Return:** The final price of the order.
- **String orderServeTime(string currentTime): (15 marks)**
    - **Parameters:**
        - `currentTime:` Time is in 24 hr format e.g. "13:00"
    - **Purpose:** Computes and returns the estimated time the order will be served. It may be assumed that the chef can only prepare one item at a time. (Remember to handle edge cases passing midnight ☺. Zaan runs a 24hr service on popular demand for LUMS students ☺)
    - **Return:** Time at which meal will be served (in 24 hr format e.g. "14:30")
- **void splitOrder(Order &newOrder, int splitIndex) (10 marks)**
    - **Parameters:**
        - `newOrder:` (object already declared for you) represents an empty new order object.
        - `splitIndex:` the index from which your order of menuItems should be splitted between your order and neworder.
    - **Purpose:** Allows 2 customers to split their order bills after placing a single order.
        - The menuItems inclusive and after the splitIndex will be added to newOrder.
        - These menuItems will then be removed from your current Order object.
        - Updates `itemCount` of both Order objects correctly.
- **void printOrderDetails() (10 marks)**
    - **Purpose:** Prints the order details in the form of a receipt. Details include
        - customer name
        - customer contact number
        - list of ordered items (with their individual prices)

- total price payable (without discount),
- time at which order was placed
- time at which order was served.

- **Destructor: prints a message when an object of type Order is deleted**

End of Lab 06